

The Brokerage Playground

How the desk tool turns a balance sheet into goals, ideas into client-specific recommendations, and research into a scored daily focus — every formula, end to end.

Prepared	Audience	Source of truth	Nature
18 June 2026	The detail-oriented reader who needs the ins and outs	data.js · goals.js · mapping.js · scanner.js · charts.js · build_today_focus.py · app.js	Static client-side app · no build step · all logic is pure functions

The one-sentence version. The Brokerage Playground is a private-bank desk tool that **infers** each client's goals from their balance sheet, **derives** (never hand-picks) which investment ideas fit which clients through a single transparent scoring engine, and **scores** each idea's standalone conviction from a research feed — so every recommendation on screen can be opened up and traced back to a number and a reason.

This document is exhaustive by design. Section 1 sets out the goal of the project and its mental model. Sections 2–3 cover the data the whole thing runs on. Sections 4–8 are the engines, with every calculation written out. Sections 9–11 cover where ideas come from, how they are tagged to clients, and the supporting analytics. Every formula here is copied from the live code, not paraphrased.

Contents

1. The goal of the project
2. The four tabs — what the user sees
3. The data model
4. Goal inference — `deriveGoals()`
5. The unified idea→client fit engine
6. The four fit axes, in full
7. The tradability gate & goal-alignment multiplier
8. Conviction scoring — two rubrics
9. Where ideas come from
10. The portfolio scanner — 11 rules
11. Supporting analytics & pre-trade
12. Worked end-to-end example
13. Calibration, limitations & honesty

1 · The goal of the project

The Brokerage Playground is a **client-focused desk tool** for a private-bank investment advisor, restyled in a **J.P. Morgan Private Bank** aesthetic (espresso and bronze, serif display type, hairline rules). It is a static website — no server, no build step, no database. Everything is a **pure function over seed data** held in the browser, which is the whole point: every recommendation is **reproducible and inspectable**, never a black box and never hand-curated.

It exists to answer four questions an advisor faces every morning, and to answer them *transparently*:

1. **What does this client actually want?** — not what a form says, but what their balance sheet and behaviour imply. (Goal inference.)
2. **Of the desk's standing ideas, which fit *this* client, and how well?** — with a per-axis breakdown you can open up. (The fit engine.)
3. **Of today's market events, which are worth acting on, and for whom?** — a daily sweep scored for conviction and mapped to the live book. (Today's Focus.)
4. **If I stage this trade, is it suitable, fundable and on-plan?** (Pre-trade analysis.)

The governing principle: *idea→client links are derived, never hand-picked, so the views can't drift.* If you change a client's book, the clients an idea is "flagged" to recompute automatically — there is no hand-maintained mapping table that can fall out of sync with reality.

Two distinct axes the tool keeps separate

A recurring source of confusion in advice tools is conflating "is this a good idea?" with "is this idea right for this client?" The Playground keeps them on **two orthogonal axes**:

Axis	Question	Range	Computed by
Conviction	How good is the idea, on its own merits?	0–100 + tier	<code>build_today_focus.py</code> (rubric)
Client-fit	How right is this idea for <i>this</i> client's book and goals?	0–100 + tier	<code>mapping.js</code> (4 axes × gate)

A high-conviction idea can be a poor fit for a conservative book; a modest idea can be exactly what an under-allocated client needs. The two scores are shown side by side and never blended into one misleading number.

2 • The four tabs — what the user sees

2.1 Solutions Views

The desk's standing investment **themes** on a left rail (AI & Productivity, Power & Infrastructure, Core Fixed Income, ...). Selecting a theme filters its idea tiles. Opening an idea shows **how the desk would express it** (clickable expressions / structures) and **which client books it fits** — that client list is produced live by the fit engine, not stored. The tab also offers **Draft a view**: type a one-line thesis and the app suggests a theme, asset class, expressions and candidate client books to keep or edit, persisting the result to `localStorage` (§9.3).

2.2 Today's Focus

An AI-generated daily market sweep, split into **Earnings** and **Ex-earnings** sections. Every idea card carries a **conviction score** (the rubric of §8), the **clients it is flagged to** — each with its own **client-fit score** and a one-line *why* — clickable expressions, and, for earnings, the report date, implied-vs-historical move and the conviction pillars. A tag shows whether the idea leans on a Solutions Views theme or is **off-theme** (with the reason). The data is generated from a file, not hand-coded into the app (§9.2).

2.3 Advisor Book

Your whole book as line items, with a client dropdown. Each client opens to their portfolio: the allocation-vs-goal-target bar, the desk's agenda, the ideas mapped to them with reasoning, and top holdings. The "how were these goals derived" panel exposes the full goal-inference arithmetic (§4).

2.4 Pre-Trade Analysis

Stage a trade (client × idea × structure × size) for a first-pass read on suitability, funding, concentration, currency and the desk view (§11.2).

3 · The data model DATA.JS

Three collections seed everything: **themes**, **ideas**, and **clients**. They are exposed on `window.SEED`. Understanding their fields is a prerequisite for every formula that follows.

3.1 Themes

Nine standing themes (`SEED_THEMES`), each `{ id, name, blurb }`: AI & Productivity, Power & Infrastructure, Core Fixed Income, Broadening Equity, Real Assets, Resilience & Protection, Gold & Currency, Healthcare Innovation, Structured Outcomes. Themes are the "house view" buckets; an idea's theme membership later feeds the conviction house-view pillar.

3.2 Ideas

Each idea (`SEED_IDEAS`) is tagged on four independent dimensions plus copy:

Field	Values	Role
<code>type</code>	Thematic · Opportunistic · Strategic	The idea's character / horizon style
<code>assetClass</code>	Equity, Fixed Income, Real Assets, Commodity, Alternatives, Structured, Multi-Asset	What instrument class it lives in
<code>sector</code>	Technology, Utilities, Energy, Gold, Rates, Credit, Broad, ...	Sector signal for the fit axes
<code>bucket</code>	Growth Income Preservation	Which of the three goal buckets the idea <i>serves</i>
<code>conviction</code>	High · Medium-High · Medium	The desk's standing conviction (Views ideas)
<code>structures</code>	e.g. ["Direct equity", "Zero-cost collar", "Buffered note"]	How to express it; <code>structures[0]</code> is the <i>natural expression</i>
<code>intent</code>	add · protect · trim · income	Legacy/display field; no longer drives the engine
<code>tickers</code>	optional alias list, e.g. gold = [XAU, GLD, 4GLD, ...]	Lets the engine recognise the idea in a book by name

The seed carries roughly 17 ideas across the nine themes — from "AI infrastructure & compute leaders" (Equity / Technology / Growth) to "Gold as a debasement hedge" (Commodity / Gold /

Preservation) to the desk's flagship structured note, a HALO equal-weight basket autocall paying a fixed 24% p.a. coupon with a 20% cushion.

3.3 Clients — the seven books

Seven deliberately different books (`SEED_CLIENTS`), each engineered so the scanner and the fit engine surface a *different* set of actions. Each carries: `aum` , `ccy` (base currency), `classification` (Retail / Professional) and `mifid` , a free-text `risk` string, a `split` (asset-class weights), an array of `positions` , optional `liabilities` , and a `goals` object holding a free-text objective, horizon, and (sometimes) a `funding` headline.

Client	AUM	Class	Stated risk	The teaching point
Fable	\$38m	Professional	Aggressive	Concentrated AI book, options-fluent
Aurora (real anchor)	€50.2m	Retail	Growth + income	25.8% Micron on a ~10× gain; 72% USD vs EUR base
Scott	\$124m	Retail	Conservative income	Large diversified retirement/preservation estate
Amar	\$71m	Professional	Growth, real-asset tilt	Real assets + a 16% Bitcoin position down 38%
Jacob	\$96m	Professional	Moderate	Balanced book + a \$19m mortgage & tax bill (liabilities)
Prahnav	\$44m	Retail	Growth	Buckets on-target, but two concentrated winners; Retail collar problem
Ben	\$29m	Retail	Moderate, value tilt	82% equity, thin income/protection
Mitch (was Morgan)	\$38m	Professional	Aggressive — max growth	No funding goal & no targets — goal must be inferred from the book
Fotis (was Tejpaal)	\$60m	Retail	Not profiled	No mandate, no goal, never risk-profiled — read purely off the book

Mitch and Fotis are the deliberate "nothing on file" cases that prove the goal-inference engine earns its keep. Aurora is the real anchor book; the others are consistent, distinct private-bank books.

3.4 The reconciliation invariant

`positions` is the single source of truth. Each book's `weightPct` values **sum to 100**, and each client's `split` literal **equals the per-asset-class sum of its positions** (`split` $\equiv \sum$ `positions`). So whether a calculation reads the `split` or folds the `positions`, it sees the same book.

3.5 Synthetic 24-month sector history

The books only carry *current* holdings, but the Affinity axis (§6.2) needs history. So `data.js` synthesises, per client, a **24-month monthly sector-allocation series** (`client.sectorHistory[sector]` , index 0 = most recent = current). It is **deterministic** (no RNG) and **clearly labelled synthetic seed data**. The trajectory is shaped from each sector's dominant-position P&L as a proxy for how the weight got there:

Shape	Trigger (top position P&L)	Trajectory toward "now"	Example
ramp	winner, $pnl \geq +50\%$	starts low (~12% of current), rises into the book	Micron in Aurora
fade	loser, $pnl \leq -10\%$	starts high (~130% of current), drifts down	an underwater bond
core	else	roughly steady around current (~92% start)	an index core

Invariant: `history[0]` exactly equals the live current allocation, so the synthetic past is always consistent with the real present. To go live, replace the array with a true monthly series — nothing else changes.

3.6 MiFID complexity classification

The single most consequential domain rule. Every structure string is classified by `complexityOf()` into one of three buckets via keyword match — **order matters** (structured is checked before OTC, so "HALO basket (ACM+)" reads as a note, not a plain basket):

Class	Examples	Retail can trade?
structured	autocall, Phoenix, reverse convertible, buffered note, capital-protected note, certificate, HALO	Yes — a packaged <i>security</i> (still complex → appropriateness applies)
otc	collar, risk reversal, accumulator, prepaid variable forward, FX forward, covered call, protective put	No — a bilateral OTC <i>derivative</i> ; needs Professional re-classification
non-complex	direct equity, ETF, index core, bond ladder, fund	Yes

This single classifier drives the global **tradability gate** (§7.1) and the pre-trade appropriateness check (§11.2). The distinction it encodes — packaged structured *notes* are Retail-eligible, OTC *derivatives* are not — is the heart of the app's suitability logic.

4 • Goal inference — `deriveGoals()` `GOALS.JS`

Real private-bank clients never hand you a clean target allocation. They say "I want to grow capital," "I have a \$19m mortgage," "I need \$0.8m/yr." The Playground therefore **infers** a goal from *both sides of the balance sheet* plus revealed risk appetite, into three buckets:

PRESERVATION **INCOME** **GROWTH**

The methodology is liability-driven / goals-based (in the spirit of Chhabra's risk-allocation framework): **fund the floor first** (obligations are not preferences), then **deploy the surplus** by how hard the money must work. Every coefficient is a calibration choice and lives in `PARAMS`; every *input* is a real measured quantity off the balance sheet.

Anti-circularity (the crucial design choice). The current book sets only a *willingness knob* (a single scalar), **not** the target vector. The target is still built bottom-up by the bucket logic, so the gap between what the client *has* and what they *need* stays meaningful. Without this, the model would always report "you're on target."

4.1 Step 1 — measure the inputs off the balance sheet

Horizon (`parseHorizon`)

Parses `goals.horizon` for a phase and a midpoint. `/drawdown|decumulat|retire/` → `drawdown` phase, else `accumulation`. Two numbers → their average as years; one → that number; none → default 7 years.

Funding goal → required return and/or income need (`parseFunding`)

Handles three headline shapes. An income unit (`".../yr"`) or a `"$X m/yr"` match yields an **annual income draw**. A `"to $X m"` match yields a **terminal value target**. The required return is then a compound-growth solve:

$$\text{requiredReturn \%} = ((\text{terminalTarget} / \text{baseValue}) ^ { (1 / \text{years}) - 1 }) \times 100$$

- `baseValue` = explicit `funding.current` for value goals, else AUM.
- `years` = an explicit "by 20XX" minus 2026 (the env year), else the horizon midpoint.
- Only computed when `terminalTarget > baseValue` (no negative required returns).

Liabilities → near-term bullets vs serviceable debt (`parseLiabilities`)

Each liability is classified by name/note keywords and summed, all as a **% of AUM**:

Quantity	Trigger	Feeds
<code>nearBillPct</code>	<code>near:true</code> or tax / due / bridge / bill / payable / Q1–4 / settlement	Preservation reserve
<code>servicePct</code>	mortgage / loan / line / credit / leverage / margin / borrow → <code>amount × rate</code> (rate default 6%)	Income (debt servicing)
<code>debtLoadPct</code>	every liability's amount (total leverage)	Income & Preservation matching/ballast

Willingness — stated and revealed (the two-way read)

Stated (statedWillingness) parses the free-text risk string to a 0–1 scalar: aggressive/max → 0.95, growth → 0.80, moderate/balanced → 0.50, conservative/income/preservation → 0.30, unparseable → null .

Revealed (revealedWillingness) reads it off the *current book*, three signals blended:

R = share of book in risk-on assets (Equity + Alternatives)clamp 0–1

D = 1 – min(1, defensiveShare / 0.55)defensiveShare = Cash+FixedIncome+Commodity

C = min(1, (largestGenuineSingleName% / 100) / 0.25)concentration appetite

revealed willingness = 0.5·R + 0.3·D + 0.2·C(clamped 0–1)

The blended willingness is 0.5·stated + 0.5·revealed when a risk profile is stated, and purely revealed when it isn't (Mitch, Fotis). The "largest genuine single name" excludes index cores and fundless sleeves (no ticker / sector "Broad").

4.2 Step 2 — build the three raw bucket scores

Every bucket starts at a small base = 8 and accumulates named, inspectable terms. Let caution = 1 – willingness , willingCoef = 30 when a required-return signal exists, else 45 (lean harder on willingness when there is no written goal), and concExcess = max(0, topName% – 15) .

Bucket	Raw score = sum of these terms
Preservation	8 + 1.0 × nearBillPct + 0.3 × debtLoadPct (ballast vs leverage) + 1.0 × concExcess (single-name > 15%) + 15 if drawdown phase + 3 × max(0, 5 – horizonYears) + 28 × caution (temperament ballast)
Income	8 + 8 × incomeNeedPct (capitalises the draw) + 8 × servicePct (debt service) + 0.6 × debtLoadPct (income-match the debt) + 15 if drawdown phase + 18 × caution (stable-cash preference)
Growth	8 + 5 × requiredReturn% + 3 × max(0, horizonYears – 6) + willingCoef × willingness

Willingness is bidirectional. A low appetite must *add ballast*, not merely shrink Growth — otherwise a client with no liabilities and no income need comes out growth-heavy no matter how cautious they are (Income/Preservation would only ever get weight from needs). The caution = 1 – willingness terms on Preservation (×28) and Income (×18) fix this.

4.3 Step 3 — normalise to exactly 100

The three raw scores are normalised with **largest-remainder rounding** (normalizeTo100) so the vector always sums to *exactly* 100 with integer parts — floor each scaled value, then hand the leftover points to the largest fractional remainders. This avoids the "99% / 101%" artefacts of naive rounding.

4.4 Provenance — source & confidence

Condition	source	confidence
A funding goal yields a required return	stated-goal	0.90
No goal, but a risk profile is stated	stated-risk	0.70
Nothing on file — read purely off the book	revealed	0.55

A human-readable `drivers[]` list is assembled (e.g. "Needs ~6.3%/yr over 9y to hit the funding goal → Growth"; "25.8% single-name concentration → Preservation") and surfaced verbatim in the Advisor Book's "How were these goals derived" panel. A `client.goalOverride` pins a manual vector and short-circuits the whole model (source "override", confidence 1).

4.5 Folding the *current* book into the same three buckets

To draw the "now vs goal" bar, the live book is folded into the same three buckets by **role** (`currentBuckets`), so it reconciles with the derived goal:

Asset class	Folds into
Equity, Alternatives (incl. crypto, venture)	Growth
Fixed Income, Credit, Real Assets	Income
Cash, Commodity (gold)	Preservation
Structured notes	by <i>purpose</i> : buffer/protect/capital → Preservation; autocall/coupon/income → Income; else Growth

There is no separate "Structured" or "Liquidity" goal bucket; cash and gold are ballast (Preservation), and structured notes are classified by what they do.

5 • The unified idea→client fit engine MAPPING.JS

There is **one** fit scorer: `window.MAPPING.scoreIdeaForClient(idea, client)`. Everything funnels through it — Today's Focus flagging, the Advisor-Book top-3, the Draft-a-view preview, and the Solutions Views client lists. In `scanner.js`, `ideaFit` / `clientsForIdea` / `matchedIdeas` are **thin facades** that call it and keep their legacy output keys. So the saved-View client list and the draft-preview list can never disagree — same algorithm, same gate.

5.1 The shape of the calculation

Client-fit is a **gated, goal-aligned weighted sum of four transparent axes**, each scored 0–100 with a plain-English note. In one line:

$$\text{Final Client-Fit} = \text{Tradability} \times \text{GoalAlignment} \times \Sigma(\text{axis}_i.\text{score} \times \text{weight}_i)$$

where $\Sigma \text{weight}_i = 1.00$ (so the bracketed sum stays 0–100)
Tradability $\in \{0, 1\}$ binary MiFID gate (§7.1)
GoalAlignment $\in [0.85, 1.15]$ multiplier (§7.2)

5.2 The four axes and their flat weights

Weights are **flat** (fixed per axis), held in `PARAMS.weights`. They are the original five-axis weights with House-view fit (0.15) removed and the remaining four rescaled by $1 / 0.85$ so they keep their relative balance and sum to exactly 1.00. The **exact fractions** are used in the math; the UI shows rounded values.

Axis	What it measures	Exact weight	Shown
Affinity fit holdings	Recency-weighted sector affinity – over-limit penalty	$0.25 / 0.85 = 0.2941\dots$	0.29
Mandate & risk mandate	$0.6 \cdot \text{RiskSuitability} + 0.4 \cdot \text{IntentFit}$	$0.25 / 0.85 = 0.2941\dots$	0.29
Gap fit gap	Headroom toward the client's goal-bucket target	$0.20 / 0.85 = 0.2353\dots$	0.24
Concentration within sector concSector	$(1 - \text{Herfindahl}) \times 100$, inverted for fit	$0.15 / 0.85 = 0.1765\dots$	0.18
Total		1.0000	≈1.00

5.3 Tiering and gates

Output	Rule
Tier	$\text{fit} \geq 66 \rightarrow \text{Strong} \cdot \geq 48 \rightarrow \text{Good} \cdot \text{else } \text{Marginal}$
applies (Views / draft)	tradable and aligned weighted sum ≥ 45 (<code>applyMin</code>)
flagClients (Today's Focus)	$\text{fit} \geq 50$ (<code>flagMin</code>), top 6 (<code>flagMax</code>)

why	the suppression reason when gated, else the highest- <i>contribution</i> axis's note
-----	--

Suppressed (non-tradable) clients have $\text{fit} = 0$, so they fall out of every flag/apply list, but the UI **surfaces them separately** with the MiFID reason rather than silently dropping them (`suppressedClients(idea)`).

5.4 Idea descriptors — explicit, with derived fallbacks

The axes read a few derived descriptors when an idea doesn't carry them explicitly:

- **naturalExpression** = `structures[0]` if absent. The expression the tradability gate tests.
- **goalTypeOf** → appreciation | yield | protection. From the bucket, refined for Structured ideas by scanning the structures/title text.
- **riskProfileOf** → {vol, beta, structured}. `structured` from asset-class/structures; `beta` from the sector (HIGH_BETA = Technology, Crypto, Materials, Energy, Industrials, Consumer; LOW_BETA = Utilities, Gold, Rates, Credit, Infrastructure, Real Estate, FX; else moderate); `vol` from bucket + beta + structured.

6 • The four fit axes, in full

All constants live in `PARAMS`. The axes operate on **sector-level** signals (sector exposure, the 24-month history, in-sector holdings) plus the client's parsed mandate. The client's **mandate class** — growth / income / preservation — is derived from the parsed risk string (`mandateClass`), and the **strategic sector peg** is a single source of truth (`PARAMS.affinity.comfort = {growth:25, income:15, preservation:10}`), read by `sectorPeg(client, sector)`.

6.1 Gap fit — bucket-aware (weight 0.24)

Does the idea fill a *goal bucket* the client is under on? It pegs the idea's bucket to the client's own **derived goal vector**, not the sector ceiling:

$$\text{Gap fit} = \max(0, (\text{bucketTarget} - \text{bucketCurrent}) / \text{bucketTarget} \times 100)$$

`bucketTarget = goalsFor(client)[idea.bucket]` (the derived goal, \$4)

`bucketCurrent = currentBuckets(client)[idea.bucket]` (the live book, \$4.5)

So a Preservation idea (gold) scores **0 once the client's Preservation bucket is at/over target**, rather than being judged against a sector ceiling. Worked (Preservation goal 35%): current 10 → 71, 25 → 29, 35 → 0. **Fallback:** if the client has no goal target for that bucket, it reverts to sector headroom vs the mandate comfort peg, $\max(0, (\text{peg} - \text{sectorCurrent}) / \text{peg} \times 100)$.

Live example. Gold × Amar: his Preservation bucket sits at 35% against a 35% derived goal → Gap fit 0 (no headroom), correctly dropping his overall fit from a misleading "Strong" to "Good." Clients under their Preservation target score high on the same idea.

6.2 Affinity fit (weight 0.29)

$$\text{Affinity fit} = \max(0, \text{ThematicAffinity} - \text{ConcentrationPenalty})$$

A • Thematic Affinity (0–100) — a recency-weighted percentile of the current sector allocation within the 24-month history. With decay $\lambda = 0.94$ (month t weight = 0.94^t , $t=0$ most recent):

$$\text{ThematicAffinity} = \left(\sum_t w_t \cdot [\text{history}_t \leq \text{current}] \right) / \left(\sum_t w_t \right) \times 100, \quad w_t = 0.94^t$$

i.e. "how high in its own trailing-24-month range is this book's current allocation, weighting recent months more." Edge cases: no current exposure → 0; holds the sector but no history on file → treated as steady → ~100.

B • Concentration Penalty (subtracted) — how far the current allocation overshoots the mandate's sector comfort peg:

$$\text{Penalty} = \max(0, (\text{current} - \text{sectorPeg}) \times 10, \quad \text{capped at } 100)$$

Affinity is sector-based; Gap fit is now bucket-based, so the two no longer share one peg — only the Affinity penalty reads `sectorPeg`.

6.3 Mandate & risk (weight 0.29)

$$\text{Mandate \& Risk} = 0.6 \times \text{RiskSuitability} + 0.4 \times \text{IntentFit}$$

(Tradability is *not* on this axis any more — it is a global gate, \$7.1, so keeping it here would double-count.)

Risk Suitability (0–100 + reason) — a deterministic matrix of the idea's {vol, beta, structured} against the mandate. Highlights:

Mandate	High-beta/high-vol	Moderate	Low-vol	Capital-protected
growth	92	72	55	—
income	45	82	90	—
preservation	22	50	80	92

Intent Fit (0–100 + reason) — a matrix of the idea's goal type against the mandate's goal (`INTENT_FIT[mandate][goalType]`):

Mandate ↓ / Goal type →	appreciation	yield	protection
growth	90	55	45
income	60	90	65
preservation	30	68	92

Worked. Income client + a low-vol dividend equity (`vol:low` , `goalType:appreciation`): Risk Suitability 90, Intent Fit 60 → $0.6 \cdot 90 + 0.4 \cdot 60 = 78$.

6.4 Concentration within sector (weight 0.18)

A Herfindahl diversification score of the book's holdings *inside the idea's sector*:

$$\begin{aligned} \text{HHI} &= \sum (w_i)^2 && \text{over in-sector holdings, weights normalised to sum to 1} \\ \text{raw} &= (1 - \text{HHI}) \times 100 && \text{concentrated} \rightarrow 0, \text{ diversified} \rightarrow 100 \end{aligned}$$

Worked: one name → HHI 1.0 → raw 0; five equal names → HHI 0.20 → raw 80.

Inverted for fit by default. A concentrated sector position *needs* a new name, so it should fit **more**: `fitContribution = invertForFit ? (100 - raw) : raw`, controlled by the single flippable line `PARAMS.concWithinSector.invertForFit` (default `true`). The breakdown shows both the raw diversification score and the fit contribution. No in-sector holdings → neutral 50.

Live example. Fable's six spread Technology names → HHI ≈ 0.19, diversification 81, inverted fit contribution **19** — already diversified within tech, so a new tech name adds little.

6.5 Why House-view fit is *not* a fit axis

House-view fit was removed from client-fit because it is an **idea-level property**, not a per-client signal, and it double-counted holdings already captured by Affinity and Gap fit. It now lives in the **conviction** score instead (§8), as a pillar of the ex-earnings model. The on-theme / off-theme tag still shows on every tile.

7 • The gate and the multiplier

7.1 The global tradability gate (binary, MiFID)

Computed **once** per idea×client, *before* the weighted sum, and multiplied across the whole score. Not an axis — it used to be one term on Mandate & Risk, but that let a non-tradable idea still score up to ~75; moving it to a global gate means it fully suppresses the idea.

```
tradability(idea, client) =  
  0  if client.classification == "Retail" AND  
  isOtcOption(naturalExpression)  
  1  otherwise          (Professional, or a non-complex / structured-note  
                          expression)
```

A 0 $\Rightarrow$ fit = 0, suppressed = true, applies = false, and the reason ("Not tradable — Retail classification doesn't permit ... (OTC derivative)...") is surfaced in the UI. Because the four weights already sum to 1.00, the gate only ever passes the bracket through ($\times 1$) or zeroes it ($\times 0$); nothing is re-normalised.

Live example. Aurora (Retail) + a zero-cost collar (OTC) $\rightarrow$ tradability 0 $\rightarrow$ fit 0, suppressed with the MiFID reason. A Professional, or a structured-note expression, $\rightarrow$ 1 and the score passes through.

7.2 The goal-alignment multiplier (0.85–1.15)

A goals-aware nudge over the bracketed score (`goals.js::goalAlignment`). It reads the client's derived 3-bucket goal and boosts an idea that fills a bucket the client is *under* on, trims one that piles into a bucket already *over*:

```
b    = goal bucket the idea serves (goalBucketOfIdea)  
gap  = goalsFor(client)[b] - currentBuckets(client)[b]      (+ = under-  
allocated)  
mult = clamp( 1 + gap / 200, 0.85, 1.15 )
```

This is the signal that makes Today's Focus, Views and the Advisor Book all converge on the clients whose *goals* an idea actually serves. (Gap fit, §6.1, captures the same intuition as an axis; the multiplier applies it once more across the whole score, deliberately bounded so it nudges rather than dominates.)

7.3 Putting it together — `scoreIdeaForClient`

```
bracketFit  = round(  $\sum$  axisi.score  $\times$  weighti )      # four axes,  $\sum$ weight =  
1.00  $\rightarrow$  0–100  
alignedFit  = bracketFit  $\times$  goalAlignment.mult          # 0.85–1.15  
fit         = tradable ? round(alignedFit) : 0           # binary MiFID gate  
tier        = fit  $\geq$  66 Strong  $\cdot$   $\geq$  48 Good  $\cdot$  else Marginal
```

The returned object is a superset every call site consumes: { fit, tier, why, axes[4], bracketFit, tradable, suppressed, tradabilityReason, naturalExpression, goalAlignment, applies, score, reason, gap, secExp, acExp }. Each axes[] row is { key, label, weight, score, contribution, note } — the per-axis breakdown that opens on click, so the reasoning is never a black box.

8 · Conviction scoring — two rubrics

BUILD_TODAY_FOCUS.PY

Conviction is the idea's *standalone* quality, computed by the generator (not the browser) from the research payload. There are **two rubrics, chosen by `idea.kind`**. Each pillar carries its own `max` and a data-quality tag `dq` $\in$ {sourced, estimated, unverified}. The generator stamps labels/maxes, sums the raw score, scales to /100 and bands it.

8.1 Earnings ideas — the print rubric (/8)

Four pillars, each **0–2**:

Pillar	0–2 criterion
Asymmetry signal	implied straddle move $\div$ avg absolute realised move over the trailing 4 prints — materially <1 (market underpricing) $\rightarrow 2$ · near fair $\rightarrow 1$ · implied rich $\rightarrow 0$
Sell-side consensus	majority buy / upside to mean PT / positive last-30-day revisions — all three $\rightarrow 2$ · partial $\rightarrow 1$ · none/contradicts $\rightarrow 0$
Catalyst clarity	pre-print: a dated, unpriced catalyst; post-print: a reaction disproportionate to print quality
Positioning & sentiment	short interest $>10\%$ float OR an extreme positioning setup

The **banding** governs the label directly (not a simple /100 cut):

High	raw ≥ 6 , no zero pillar, all inputs sourced
High – data gap	same, but ≥ 1 estimated input
Medium	raw 4–5, OR any zero pillar, OR consensus unverified
Low	raw < 4 , OR two-plus zero pillars (excluded)

8.2 Ex-earnings ideas — the seven-pillar model (/15)

Pillar	Max	What it scores
Gap / asymmetry	3	thesis expected move (entry $\rightarrow$ target) $\div$ the name's normal move: $\geq 2.0 \rightarrow 3$ · $1.5-2.0 \rightarrow 2$ · $1.0-1.5 \rightarrow 1$ · $<1.0 \rightarrow 0$
Catalyst	2	a dated, hard reason it moves soon (earnings, FOMC, CPI, ECB)
Consensus + confirmation	2	sell-side alignment <i>or</i> a defensible variant, AND an independent signal (breakout/flow/skew)
Fundamental thesis	2	two 0/1 checks: driver specificity (named, quantified) + variant view (computable gap vs consensus)
House-view fit	2	core desk theme / direct $\rightarrow 2$ · on-theme / adjacent $\rightarrow 1$ · off-theme / against $\rightarrow 0$

Positioning	2	confirmation/veto: SI >10% float, extreme CoT, fund flows + RSI sentiment read
Technical	2	confirmation/veto: 50/100/200-day MA alignment, Bollinger entry, Fibonacci off the 6-mo swing
Total	15	

score = raw ÷ 15 × 100 High ≥ 75 • Medium ≥ 55 • Watch ≥ 0

The view drives; technicals confirm — roughly 70/30. Fundamental thesis + house view + the thesis-bearing pillars (Asymmetry, Catalyst, Consensus) carry **11/15 ≈ 73%**; Positioning + Technical carry **4/15 ≈ 27%** and are framed as *confirmation / veto, not drivers* — they can support or kill a view but can't manufacture conviction. RSI lives only in Positioning; moving averages only in Technical — no overlap.

8.3 The data-quality cap (global modifier)

If *any* pillar's core input is `estimated` or `unverified` (`capped = true`), the ex-earnings label cannot exceed **Medium**; the earnings model expresses the same idea through the "High — data gap" label. With the current playground data (straddle / short-interest / RSI feeds not wired up, so those pillars are `estimated`), the cap is binding on every idea — wire sourced inputs in and Highs unlock.

8.4 `changeMyMind` — stated but not scored

Every idea — earnings and ex-earnings — carries a `changeMyMind` field: the condition that would invalidate the view. It is surfaced on the conviction tile but **deliberately not scored**, since a discrete kill-switch would bias the score toward dated catalyst trades over strategic holds.

9 • Where ideas come from

Ideas enter the system through **three distinct channels**, all tagged into the same four-dimension schema (type / asset class / sector / bucket) so the downstream engines treat them identically.

9.1 Channel A — the standing Solutions Views board

The ~17 seed ideas in `data.js` are the desk's **standing themes**, synthesised around a J.P. Morgan Private Bank mid-year-outlook positioning (AI & productivity, power & infrastructure, extending duration, broadening beyond mega-caps, real assets, resilience/protection, gold & currency, healthcare, structured outcomes). The copy is original; each idea is hand-authored once with its `thesis`, `conviction` and `structures`, then **everything client-facing is derived** — which clients it applies to is computed, never typed.

9.2 Channel B — the daily market sweep (Today's Focus)

Today's Focus is **schedule-ready**: it is generated from a data file, not hand-coded into the app. The pipeline:

Step	File	What happens
1 • Sweep	<code>today_focus.json</code>	A Claude Code routine sweeps the market (Reuters, WSJ, Bloomberg, Yahoo, Seeking Alpha, Barron's), applies a ≥2-source rule , tags every fact <code>sourced</code> / <code>estimated</code> , authors the conviction pillar scores + <code>dq</code> tags, and overwrites the raw payload.
2 • Build	<code>build_today_focus.py</code>	Validates (≥2 sources each, facts tagged, no forward event described in past tense), applies the conviction rubric (§8), and writes the app file.
3 • Load	<code>today_focus.js</code>	<code>window.TODAY_FOCUS</code> — what the app loads. Auto-generated; never edited by hand.

A daily scheduled run does steps 1–3 and commits; **no app edits are needed**, and because the client mapping is computed in the browser at render time, the flagged clients always reflect the current book.

The shape of one swept idea

Each Today's Focus idea carries: `identity` (`id`, `name`, `ticker`, `sector`, `assetClass`, `bucket`), a headline and `thesis`, its `themeId` (or `offThemeWhy`), `structures`, `changeMyMind`, the `conviction.pillars[]` (each `{key, score, dq, note}`), an `earnings` block (report date/time, implied vs historical move, what to watch) for earnings ideas, a tagged `facts[]` list, and a `sources[]` list. The generator validates all of it and refuses to silently pass — it prints warnings for any idea with fewer than two sources, an untagged fact, or a report date before `asOf`.

9.3 Channel C — Draft a view (advisor-authored, on the fly)

The advisor types a one-line thesis; `draftFromThesis()` matches it against a keyword table (`DRAFT_RULES`) to suggest a theme, sector, asset class, bucket and expressions:

Keywords (any match)	→ Theme	Sector / Class / Bucket
ai, semiconductor, chip, gpu, compute, hbm, nvidia, datacenter, software	AI & Productivity	Technology / Equity / Growth
power, grid, electric, utility, nuclear, smr	Power & Infra	Utilities / Equity / Income
bond, duration, yield, treasury, rates, fixed income	Core Fixed Income	Rates / Fixed Income / Income
gold, bullion, debasement	Gold & Currency	Gold / Commodity / Preservation
protect, hedge, downside, collar, buffer	Resilience & Protection	Broad / Multi-Asset / Preservation
health, glp, pharma, medtech, biotech, device	Healthcare Innovation	Healthcare / Equity / Growth
coupon, autocall, structured note, income note	Structured Outcomes	Broad / Structured
copper, materials, mining, rare earth, lithium	(off-theme)	Materials / Equity / Growth
energy, oil, gas, crude, brent	(off-theme)	Energy / Equity / Income
crypto, bitcoin, digital asset	(off-theme)	Crypto / Alternatives

The synthesised idea is then immediately scored against the live book with `MAPPING.flagClients(synth, {min: 45, max: 5})`, so the advisor sees **candidate client books with live fit scores before saving**. On save it is appended to `userData.ideas` and persisted to `localStorage`, joining the Views board exactly like a seed idea. No keyword match falls back to a generic Equity / Broad / Growth idea.

10 · The portfolio scanner — 11 rules

SCANNER.JS

Separate from the fit engine, `scanBook(client)` runs the book *the other direction*: portfolio → ideas. It is eleven hand-coded, severity-ranked rules that read the positions and surface actions. It powers the "See more ideas" asset-class list and the Next Best Action — it is **not** a per-client fit score. Each finding carries a severity (1–3), title, rationale, suggested structures, bucket, and a Retail-appropriateness flag.

#	Rule	Trigger	Sev
1	Protect concentration	single-name Equity/Alt $\geq$ 15% (not index/crypto)	2–3
2	Loss harvest	equity-like position $\leq$ -10%	2

3	Bond swap	Fixed Income $\leq -8\%$ (rates, not credit)	2
4	Rehabilitate crypto	Crypto position $\leq -20\%$	3
5	Overwrite a winner	Equity $\geq +50\%$ & $<15\%$ weight	1
6	Hedge FX mismatch	non-base, non-cash currency $\geq 40\%$ of book	2
7	Put cash to work	Cash $\geq 8\%$	1
8	Liability & cashflow planning	any liabilities present	3
9	Close the protection gap	Preservation target – current ≥ 6 pts	2
10	Lift income	Income target – current ≥ 8 pts	1
11	Diversify a sector	any sector $\geq 30\%$ (excl. Cash/Rates/Credit/Broad)	1

Rules 9 and 10 reference the *derived goal* (§4), so the scanner's protection/income gaps are consistent with the goal-inference engine. After generating findings, the scanner **re-normalises appropriateness** from the actual structures: a finding is only Retail-blocked when *every* way to express it is OTC — if a structured-note or non-complex route exists, Retail can take that instead. Findings are sorted by descending severity; the top one (else the top matched View idea) becomes the **Next Best Action**.

11 · Supporting analytics & pre-trade CHARTS.JS

· APP.JS

11.1 Allocation analytics (`charts.js`)

- **Donut / legend** — pure-SVG allocation rings from a split, in the house palette.
- **bucketAlloc(split)** — folds an asset-class split into the three goal buckets (delegates to `goals.js` so charts and pre-trade share one classifier).
- **goalTargetBar3(target, current)** — the "now vs goal" bar: the *bar* is the book's current weight in each bucket, the *notch* marks the inferred goal, and a label reads "Now X% · target Y% · $\pm\Delta$ " (on plan if $|\Delta| < 4$).
- **targetDistance(buckets, target)** = $\sum |\text{bucket}_k - \text{target}_k|$ — the L1 distance from plan, used by pre-trade to say a trade moves the book *closer to* or *further from* target.
- **fundingBar(f)** — funding-goal progress, $\min(100, \text{current}/\text{target} \times 100)$ with an On-track / Slightly-behind / Behind pill.

11.2 Pre-trade analysis (`app.js::runPretrade`)

Stage a trade — client × idea (or portfolio finding, or ad-hoc) × structure × notional %.
The engine applies the trade to a copy of the split (`applyTrade` : fund from cash first, then trim the largest non-cash sleeve), recomputes the bucket allocation and its distance from the derived goal, and emits a checklist:

Check	How it is decided
Appropriateness	Retail + OTC structure → blocked (warn); structured note → Retail-eligible but complex (ok); non-complex → cleared (ok)
Suitability	the idea's live fit reason if it applies, else a prompt to confirm fit against the mandate & objective
Funding	covered by cash, or flags how much would come from trimming an existing holding
Concentration	warns if the largest position is $\geq 20\%$ of the book
Goal alignment	moves the book ~X pts <i>closer to</i> / <i>further from</i> the strategic target (via <code>targetDistance</code>), and quotes the funding-goal status
Currency	flags an FX overlay if the book base isn't USD
Desk view	conviction · horizon · preferred expression

The result also renders before/after donuts and the goal-bucket deltas — a complete first-pass read before anything is committed.

12 · Worked end-to-end example — gold for Amar

To see the whole machine run on one idea×client pair, take "**Gold as a debasement hedge**" (Commodity / Gold / Preservation, natural expression "Physical / ETC") scored for **Amar** (Professional; ~35% of his book already in Preservation-role assets via gold + cash).

1. **Goal inference (§4)** derives Amar's three-bucket goal from his real-asset book, multi-decade horizon and growth-tilt willingness — placing his Preservation target at ~35%.
2. **Gap fit (§6.1)**: Preservation bucket current $\approx 35\%$ vs goal $35\% \rightarrow \max(0, (35-35)/35) = 0$. No headroom — gold doesn't fill a gap he has.
3. **Affinity fit (§6.2)**: his Gold allocation sits high in its own 24-month range (a "ramp" shape) $\rightarrow$ high thematic affinity, but possibly an over-peg penalty; net moderate-high.
4. **Mandate & risk (§6.3)**: gold's {vol: moderate, beta: low} against a growth mandate $\rightarrow$ middling Risk Suitability; protection goal type vs growth mandate $\rightarrow$ Intent Fit 45.
5. **Concentration within sector (§6.4)**: he holds gold + GDX miners $\rightarrow$ some diversification, modest inverted fit contribution.
6. **Bracketed sum** of the four weighted axes $\rightarrow$ a 0–100 `bracketFit`.
7. **Goal-alignment multiplier (§7.2)**: Preservation gap $\approx 0 \rightarrow$ multiplier ≈ 1.0 (no boost).
8. **Tradability gate (§7.1)**: Professional + physical gold (non-complex) $\rightarrow 1 \rightarrow$ passes.

The Gap-fit zero is what correctly demotes gold for Amar from a misleading "Strong" to a "Good" — he already *has* his protection. The exact same idea scores high for a client *under* their Preservation target (Scott, Fotis, Aurora). That is the system working as intended: **the fit follows the client's goals and book, automatically.**

13 · Calibration, limitations & honesty

The project is deliberately candid about what is and isn't real:

- **The PARAMS numbers are reasoned, not fit to outcomes.** Every coefficient (the affinity λ , the willingness weights, the axis weights, the intent matrix) is a calibration choice kept in one visible place. The in-browser harness — load the app, score against `window.SEED` — is how they were sanity-checked and where they'd be recalibrated.
- **The 24-month sector history is synthetic seed data**, clearly labelled, swappable for a real monthly series with no other change.
- **The data-quality cap is currently binding on every idea** (straddle / short-interest / RSI feeds aren't wired up), so conviction tops out at Medium until sourced inputs are connected — an honest reflection of the data on hand, not a hidden ceiling.
- **What still doesn't enter the fit:** AUM, currency-hedging depth, liquidity and liability funding (the scanner handles some of this separately). `riskProfile` / `goalType` are derived sensibly but per-idea curation would sharpen them.
- **The concentration fit-direction is a product decision** (`invertForFit`) — one flippable line, surfaced for confirmation rather than buried.



The through-line. Every number on screen — a goal vector, a client-fit score, a conviction band — is a pure function of inputs you can inspect, with a written reason attached. Nothing is hand-mapped, nothing drifts, and every claim can be opened up and traced to the formula that produced it. That transparency *is* the methodology.

Brokerage Playground — Complete Methodology Reference · Prepared 18 June 2026 · Source of truth:
data.js · goals.js · mapping.js · scanner.js · charts.js ·
build_today_focus.py · app.js. All formulas transcribed from the live code. Idea→client links are
derived, never hand-picked, so the views can't drift.